

Complete Guide

Fine-Tuning LLMs for Ollama

Using Unsloth + LoRA for Fast, Memory-Efficient Training

From raw data to a running Ollama model in 3 steps

Step-by-step instructions with complete code

Table of Contents

1. Overview & Architecture
2. Prerequisites & Installation
3. Step 1: Prepare Your Training Data
4. Step 2: Fine-Tune with Unsloth + LoRA
5. Step 3: Export to GGUF & Deploy to Ollama
6. Complete Configuration Reference
7. Supported Models
8. Troubleshooting & Tips

1. Overview & Architecture

Ollama does not support fine-tuning directly. The workflow is to fine-tune a base model using Python tools, convert it to GGUF format, and import it into Ollama.

The Pipeline

Stage	Tool	What It Does
1. Data Prep	<code>prepare_data.py</code>	Converts CSV/JSON to chat-format JSONL
2. Fine-Tune	<code>fine_tune.py</code> (Unsloth + LoRA)	Trains a LoRA adapter on your data 2x faster, 60% less VRAM
3. Export	<code>export_to_ollama.py</code>	Merges adapter, exports GGUF, creates Ollama model

Why Unsloth + LoRA?

- LoRA trains only ~1-2% of model parameters, so it needs far less memory
- Unsloth optimizes the training loop for 2x speed on the same hardware
- A 7-8B model can be fine-tuned on a single 16 GB GPU (or even 8 GB with 4-bit)
- The result is identical to full fine-tuning for most use cases

2. Prerequisites & Installation

Hardware Requirements

Model Size	Min VRAM	Recommended	Example GPUs
1B - 3B	4 GB	8 GB	RTX 3060, T4
7B - 8B	8 GB	16 GB	RTX 4070, A10
13B	16 GB	24 GB	RTX 4090, A5000
70B	48 GB	80 GB	A100, H100

Software Requirements

- Python 3.10+
- CUDA 11.8+ (for GPU training)
- Ollama installed locally (for deployment)

Installation

Install all dependencies:

```
pip install -r requirements.txt
```

Or install individually:

```
pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"  
pip install torch transformers datasets trl peft accelerate  
pip install bitsandbytes xformers sentencepiece
```

Install Ollama:

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Note: On Google Colab (free tier with T4 GPU), Unsloth works out of the box. This is the easiest way to get started if you don't have a local GPU.

3. Step 1: Prepare Your Training Data

Fine-tuning requires training data in chat format. Each example is a conversation with system, user, and assistant messages.

Option A: Start from CSV

Create a CSV file with these columns:

```
instruction,input,output
"Explain what a REST API is","", "A REST API is an architectural style..."
"Convert this to Python","for(int i=0;i<10;i++)","for i in range(10):..."
"Summarize this text","Long article here...","Brief summary here..."
```

Then convert it:

```
python prepare_data.py \
    --input your_data.csv \
    --output data/train.jsonl \
    --system-prompt "You are a helpful coding assistant."
```

Option B: Write JSONL Directly (Recommended)

Each line is a JSON array of chat messages:

```
[
  {"role": "system", "content": "You are a helpful assistant."},
  {"role": "user", "content": "How do I read a CSV in Python?"},
  {"role": "assistant", "content": "Use pandas:\n```python\nimport pandas as pd\nndf = pd.read_csv('data.csv')\n```"}
]
```

Tip: Aim for 100-1000+ examples for good results. Quality matters more than quantity. Each example should demonstrate the exact behavior you want.

Option C: Start from JSON

```
[
  {"instruction": "Explain decorators", "output": "Decorators wrap..."},
  {"instruction": "Fix this code", "input": "buggy code", "output": "fixed"}
]
```

The prepare_data.py Script (Complete Code)

```
"""Convert raw data into chat format for fine-tuning."""

import argparse
import csv
import json
from pathlib import Path

def convert_row_to_chat(row: dict, system_prompt: str) -> list[dict]:
```

```

user_content = row["instruction"]
if row.get("input"):
    user_content += f"\n\n{row['input']}"

messages = []
if system_prompt:
    messages.append({"role": "system", "content": system_prompt})
messages.append({"role": "user", "content": user_content})
messages.append({"role": "assistant", "content": row["output"]})
return messages

def load_csv(path: Path) -> list[dict]:
    with open(path, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))

def load_json(path: Path) -> list[dict]:
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    return data if isinstance(data, list) else [data]

def load_jsonl(path: Path) -> list[dict]:
    rows = []
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", required=True)
    parser.add_argument("--output", default="data/train.jsonl")
    parser.add_argument("--system-prompt",
                        default="You are a helpful assistant.")
    args = parser.parse_args()

    input_path = Path(args.input)
    suffix = input_path.suffix.lower()
    loaders = {
        ".csv": load_csv, ".json": load_json, ".jsonl": load_jsonl
    }
    rows = loaders[suffix](input_path)

    conversations = []
    for row in rows:
        # Already in chat format
        if isinstance(row, list):
            conversations.append(row)
        elif "messages" in row:
            conversations.append(row["messages"])
        else:

```

```

        conversations.append(
            convert_row_to_chat(row, args.system_prompt)
        )

    output_path = Path(args.output)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", encoding="utf-8") as f:
        for conv in conversations:
            f.write(json.dumps(conv, ensure_ascii=False) + "\n")

    print(f"Wrote {len(conversations)} examples to {output_path}")

if __name__ == "__main__":
    main()

```

4. Step 2: Fine-Tune with Unsloth + LoRA

Quick Start Command

```
python fine_tune.py \
    --model "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit" \
    --dataset data/train.jsonl \
    --epochs 3 \
    --output ./output
```

Or use a model alias:

```
python fine_tune.py \
    --model-alias llama-3.1-8b \
    --dataset data/train.jsonl \
    --epochs 3
```

Understanding the Training Script

The fine-tuning script does four things:

1. Load the base model (4-bit quantized for memory efficiency)

```
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit",
    max_seq_length=2048,
    load_in_4bit=True,
)
```

2. Add LoRA adapters (train only 1-2% of parameters)

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,                # LoRA rank: higher = more capacity
    lora_alpha=16,        # Scaling factor
    lora_dropout=0.0,     # No dropout for Unsloth
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    use_gradient_checkpointing="unsloth", # 60% less VRAM
)
```

3. Load and format the dataset

```
from datasets import Dataset
import json

# Load JSONL conversations
conversations = []
with open("data/train.jsonl") as f:
```



```

    for line in f:
        conversations.append(json.loads(line.strip()))

dataset = Dataset.from_dict({"conversations": conversations})

# Format using the model's chat template
def format_conversation(example):
    return {"text": tokenizer.apply_chat_template(
        example["conversations"],
        tokenize=False,
        add_generation_prompt=False,
    )}

dataset = dataset.map(format_conversation)

```

4. Train with SFTTrainer

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    args=TrainingArguments(
        output_dir="./output",
        num_train_epochs=3,
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        weight_decay=0.01,
        warmup_steps=5,
        lr_scheduler_type="linear",
        bf16=True,
        logging_steps=10,
        save_steps=100,
        report_to="none",
    ),
    dataset_text_field="text",
    max_seq_length=2048,
)

trainer.train()

# Save the fine-tuned adapter
model.save_pretrained("./output")
tokenizer.save_pretrained("./output")

```

Tip: Start with 3 epochs. If the model overfits (training loss drops to ~0 but output quality is bad), reduce to 1-2 epochs. If quality is poor, add more data or increase epochs to 5.

5. Step 3: Export to GGUF & Deploy to Ollama

Quick Start Command

```
python export_to_ollama.py \  
  --model-dir ./output \  
  --model-name my-custom-model \  
  --quantization q4_k_m
```

Quantization Options

Quantization	Quality	Size	Speed	Use Case
q2_k	Low	Smallest	Fastest	Experimentation only
q3_k_m	Moderate	Small	Fast	Edge devices
q4_k_m	Good	Medium	Good	Recommended default
q5_k_m	High	Medium-Large	Good	Quality-sensitive tasks
q6_k	Very High	Large	Moderate	Near-original quality
q8_0	Near-Original	Large	Moderate	Maximum quality
f16	Original	Largest	Slowest	Benchmarking only

Export Code Explained

```
from unsloth import FastLanguageModel  
  
# 1. Load the fine-tuned model  
model, tokenizer = FastLanguageModel.from_pretrained(  
    model_name="./output",  
    max_seq_length=2048,  
    load_in_4bit=True,  
)  
  
# 2. Export to GGUF (merges LoRA + quantizes)  
model.save_pretrained_gguf(  
    "./output",  
    tokenizer,  
    quantization_method="q4_k_m",  
)
```

Create the Ollama Modelfile

```
# Modelfile  
FROM ./output/model-q4_k_m.gguf  
  
PARAMETER temperature 0.7  
PARAMETER top_p 0.9
```

```
PARAMETER top_k 40
PARAMETER num_ctx 2048
PARAMETER repeat_penalty 1.1

SYSTEM """You are a helpful coding assistant
specialized in Python and data science."""
```

Create and Run the Model

```
# Create the Ollama model
ollama create my-custom-model -f Modelfile

# Run it
ollama run my-custom-model

# Test with a prompt
ollama run my-custom-model "How do I read a CSV in Python?"
```

Use from Python

```
import requests
import json

response = requests.post(
    "http://localhost:11434/api/chat",
    json={
        "model": "my-custom-model",
        "messages": [
            {"role": "user", "content": "Explain decorators"}
        ],
        "stream": False,
    },
)

print(response.json()["message"]["content"])
```

6. Complete Configuration Reference

config.py - All Training Parameters

```
from dataclasses import dataclass, field

@dataclass
class ModelConfig:
    # Model selection
    model_name: str = "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit"
    max_seq_length: int = 2048
    load_in_4bit: bool = True

    # LoRA parameters
    lora_r: int = 16          # Rank: 8, 16, 32, 64
    lora_alpha: int = 16      # Usually same as r
    lora_dropout: float = 0.0
    target_modules: list[str] = field(default_factory=lambda: [
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ])

    # Training parameters
    num_train_epochs: int = 3
    per_device_train_batch_size: int = 2
    gradient_accumulation_steps: int = 4  # Effective batch = 2*4 = 8
    learning_rate: float = 2e-4
    weight_decay: float = 0.01
    warmup_steps: int = 5
    lr_scheduler_type: str = "linear"
    bf16: bool = True          # Use bfloat16 (Ampere+)
    logging_steps: int = 10
    save_steps: int = 100
    seed: int = 42

    output_dir: str = "./output"
    dataset_path: str = "data/train.jsonl"
```

Key Parameters to Tune

Parameter	Default	When to Change
<code>lora_r</code>	16	Increase to 32/64 for complex tasks; decrease to 8 for simple tasks
<code>num_train_epochs</code>	3	Increase if underfitting; decrease if overfitting
<code>learning_rate</code>	2e-4	Lower (1e-4) for larger datasets; higher (5e-4) for small datasets

batch_size	2	Increase if you have more VRAM; decrease if OOM errors
max_seq_length	2048	Increase for long-form content; decrease to save memory

7. Supported Models

These pre-quantized 4-bit models work out of the box with the pipeline. They download automatically from HuggingFace on first use.

Alias	HuggingFace ID	Size	Best For
llama-3.1-8b	unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit	8B	General purpose, coding, reasoning
llama-3.2-3b	unsloth/Llama-3.2-3B-Instruct-bnb-4bit	3B	Fast inference, edge deployment
llama-3.2-1b	unsloth/Llama-3.2-1B-Instruct-bnb-4bit	1B	Ultra-fast, embedded devices
mistral-7b	unsloth/mistral-7b-instruct-v0.3-bnb-4bit	7B	Strong general performance
gemma-2-9b	unsloth/gemma-2-9b-it-bnb-4bit	9B	Multilingual, long context
gemma-2-2b	unsloth/gemma-2-2b-it-bnb-4bit	2B	Lightweight, fast training
phi-3.5-mini	unsloth/Phi-3.5-mini-instruct-bnb-4bit	3.8B	Reasoning, math, code
qwen-2.5-7b	unsloth/Qwen2.5-7B-Instruct-bnb-4bit	7B	Multilingual, Chinese + English
qwen-2.5-3b	unsloth/Qwen2.5-3B-Instruct-bnb-4bit	3B	Compact multilingual

Usage with alias:

```
python fine_tune.py --model-alias llama-3.2-3b --dataset data/train.jsonl
```

Usage with any HuggingFace model:

```
python fine_tune.py --model "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit"
```

Tip: Start with llama-3.2-3b for fast experimentation, then scale up to llama-3.1-8b or mistral-7b for production quality.

8. Troubleshooting & Tips

Common Errors

CUDA Out of Memory (OOM)

- Reduce batch_size to 1
- Reduce max_seq_length to 1024 or 512
- Use a smaller model (3B instead of 8B)
- Ensure gradient_checkpointing is 'unsloth'

```
python fine_tune.py --batch-size 1 --max-seq-length 1024 --model-alias llama-3.2-3b
```

Model outputs gibberish after fine-tuning

- Check training data quality - garbage in, garbage out
- Reduce epochs (you may be overfitting)
- Increase dataset size (100+ examples minimum)
- Lower learning rate to 1e-4

'ollama create' fails

- Make sure Ollama is running: **ollama serve**
- Check the GGUF file path in the Modelfile is absolute
- Try: **ollama create model-name -f ./output/Modelfile**

Best Practices

- **Data quality over quantity** - 200 high-quality examples often beat 2000 mediocre ones
- **Match the chat format** - Use the exact system/user/assistant structure
- **Be consistent** - Same style, tone, and format across all examples
- **Include edge cases** - Train on the hard cases, not just easy ones
- **Validate first** - Run prepare_data.py with --validate-only before training
- **Start small** - Test with 3B model and 50 examples before scaling up
- **Monitor loss** - Training loss should decrease smoothly; spikes = bad data
- **Use q4_k_m** - Best quality/size tradeoff for most deployments

End-to-End Example (Copy-Paste Ready)

```
# 1. Prepare data
python prepare_data.py \
  --input my_training_data.csv \
  --output data/train.jsonl \
```

```
--system-prompt "You are an expert Python developer."

# 2. Fine-tune (uses ~10 GB VRAM)
python fine_tune.py \
  --model-alias llama-3.1-8b \
  --dataset data/train.jsonl \
  --epochs 3 \
  --lr 2e-4 \
  --output ./output

# 3. Export and create Ollama model
python export_to_ollama.py \
  --model-dir ./output \
  --model-name my-python-expert \
  --quantization q4_k_m \
  --system-prompt "You are an expert Python developer."

# 4. Run it!
ollama run my-python-expert
```

All code is available in the **ollama_fine_tune/** directory of this repository.